

IT3012 / SE3062 — Practical 03: Search Agents

Student ID: IT24100532 | Branch: **Week_03** | Lecture 04: Searching & Problem Solving in AI

Part 1: Practical Implementation

Step 1.1 — Exposing the World Model

`get_percept()` in `visual_grid_game.py` now passes the global state to the agent. `agent_pos` was also added back, because a planner needs a start state (it was removed in Practical 02 to make the world partially observable).

```
'agent_pos': tuple(self.agent_pos),
'grid_size': (self.width, self.height),
'walls': list(self.walls),
'all_food': list(self.food_positions)
```

Step 1.2 — BFS, DFS and UCS

All three share the same node expansion (`successors()` returns the legal Up/Down/Left/Right moves). Only the frontier data structure changes, and each keeps a reached set/dict so it is a graph search.

```
def bfs_search(self, start, goal, walls, grid_size):
    frontier = deque([(start, [])]) # FIFO queue: shallowest node first
    reached = {start}
    while frontier:
        pos, path = frontier.popleft()
        if pos == goal:
            return path
        for action, nxt in self.successors(pos, walls, grid_size):
            if nxt not in reached:
                reached.add(nxt)
                frontier.append((nxt, path + [action]))
    return None

def dfs_search(self, start, goal, walls, grid_size):
    frontier = [(start, [])] # LIFO stack: deepest node first
    reached = set()
    while frontier:
        pos, path = frontier.pop()
        if pos == goal:
            return path
        if pos in reached:
            continue
        reached.add(pos)
        for action, nxt in self.successors(pos, walls, grid_size):
            if nxt not in reached:
                frontier.append((nxt, path + [action]))
    return None

def ucs_search(self, start, goal, walls, grid_size):
    frontier = [(0, start, [])] # priority queue ordered by g(n)
    reached = {start: 0}
    while frontier:
        cost, pos, path = heapq.heappop(frontier)
        if pos == goal:
            return path
        if cost > reached[pos]:
            continue
        for action, nxt in self.successors(pos, walls, grid_size):
            new_cost = cost + 1 # every step costs 1
            if nxt not in reached or new_cost < reached[nxt]:
                reached[nxt] = new_cost
                heapq.heappush(frontier, (new_cost, nxt, path + [action]))
    return None
```

Step 1.3 — Executing Offline Plans

SearchAgent.__init__ sets self.plan = [] and self.active_algo = 'BFS' (configurable). When the plan is empty the agent searches to the closest food (Manhattan distance, skipping unreachable pellets) and then pops one action per step. Standing on food triggers Suck.

```
def sense_and_act(self, percept):
    if percept['food_here']:
        self.plan = []
        return 'Suck'
    if not self.plan:
        start = tuple(percept['agent_pos'])
        search = {'BFS': self.bfs_search, 'DFS': self.dfs_search,
                  'UCS': self.ucs_search}[self.active_algo]
        for food in sorted(percept['all_food'], key=lambda f: manhattan(start, f)):
            path = search(start, food, percept['walls'], percept['grid_size'])
            if path:
                self.plan = path
                break
    return self.plan.pop(0) if self.plan else 'Stay'
```

The GUI has **Run BFS / Run DFS / Run UCS** buttons (all on the same map) and draws the remaining planned path as blue dots, so the routes can be compared visually. All 4 tests in test_suite.py pass (including the BFS 6-step optimal path and unreachable-goal tests).

Observation: 12x12 grid, 15 food, 300-step limit, same maps for each algorithm

Map	BFS	DFS	UCS
1	75 steps, all food	300 steps, 4 food left	75 steps, all food
2	63 steps, all food	300 steps, 4 food left	63 steps, all food
3	62 steps, all food	300 steps, 6 food left	62 steps, all food
4	66 steps, all food	300 steps, 5 food left	66 steps, all food

BFS and UCS always take the same number of steps (every step costs 1). DFS takes long winding routes and runs out of steps before clearing the map.

Part 2: Theoretical Evaluation

1. (Remember) State Space vs. Search Tree

The **state space** is the set of all possible states connected by actions. Here that is every open (x, y) cell on the grid, with moves as edges. It is a fixed graph in which each state appears only once. The **search tree** is what the algorithm builds while searching: its root is the start state and each node represents a *path* to a state. The same state can appear in many nodes (reached by different paths), so the tree can be much larger than the state space, and even infinite if paths loop.

2. (Understand) The reached set

It solves the problem of **repeated states**: redundant paths and loopy paths (cycles). A state that has already been reached is not added to the frontier again. This turns tree search into graph search. Without it, my DFS agent could step Up, then Down, then Up again forever between two cells (it always expands the newest node). It would never finish, its stack would keep growing, and it could get stuck in an infinite loop without ever finding food.

3. (Analyze) BFS vs. DFS paths

BFS uses a FIFO queue, so it expands nodes **level by level**: every path of length 1, then 2, and so on. The first time it reaches the food is therefore via the fewest steps. Every step in this grid costs 1, so fewest steps = lowest cost, which makes BFS **optimal** here (the lecture: optimal if cost = 1 per step). DFS uses a LIFO stack and always dives down the newest branch as far as it can, then returns the **first**

path it finds to the goal, not the shortest one. In the simulation this showed as long, snaking routes: DFS needed about 290 moves and still left 4-6 food, while BFS cleared all 15 food in 47-60 moves. UCS matched BFS because all step costs are equal.

4. (Evaluate) Memory on a 1000x1000 grid

Using the lecture's terms (b = branching factor, at most 4 here; d = depth of the shallowest solution; m = maximum depth):

- **BFS**: time $O(b^{d+1})$ and **space** $O(b^{d+1})$, because it keeps every generated node in memory. The whole frontier (the full "ring" of cells at the current depth) plus every reached cell is stored at once. On a 1,000,000-cell grid that can mean up to a million stored nodes, and my version stores a full action list with each node, so memory explodes before a far-away food is found. **UCS** has the same problem: $O(b^{1+C/\epsilon})$ time and space.

- **DFS**: time $O(b^m)$ but **space only** $O(bm)$, linear in depth, because it only needs the current path plus the unexpanded siblings along it. So DFS is the memory-friendly option. The trade-off is that m can be enormous on a 1000x1000 grid (a path can snake through almost every cell), and the solution it returns is not optimal.

Conclusion: the BFS/UCS bottleneck is **breadth**: the frontier grows exponentially with depth. The DFS bottleneck is **depth**: memory stays small, but the paths are long and wasteful.